BOOLEAN ALGEBRA · DIGITAL ELECTRONICS · COMPLETE NOTES

Gated Flip-Flops
Complete Study Notes

Registers · Accumulators · From Basics to Advanced

Syllabus: Gated Flip-flops · Registers · Accumulators

TABLE OF CONTENTS

01	What is a Flip-Flop? (Foundation)
02	What is a Gated Flip-Flop?
03	Gated SR Latch
04	Gated D Flip-Flop
05	JK Flip-Flop (Clocked)
06	T Flip-Flop
07	Comparison of All Flip-Flops
08	Registers
09	Accumulators
10	Quick Revision & Exam Tips

01 What is a Flip-Flop? (Foundation)

DEFINITION

A **flip-flop** is a basic **memory element** (sequential circuit) that stores **1 bit** of data. Its output depends not just on current inputs, but also on its **previous state**.

Key Property: A flip-flop has two stable states — **Q = 0** (Reset state) and **Q = 1** (Set state). It "remembers" which state it was last put into.

OUTPUTS OF A FLIP-FLOP

- **Q** — Normal output (the stored bit)
- **Q-bar (Q')** — Complemented output (always opposite of Q)
- If Q = 1, then Q' = 0 (always true — they are always complements)

COMBINATIONAL vs SEQUENTIAL CIRCUIT

Feature	Combinational Circuit	Sequential (Flip-Flop)
Output depends on	Only current inputs	Current inputs + Past state
Memory	No memory	Has memory (stores 1 bit)
Example	AND, OR, NAND gates	Flip-Flops, Registers
Clock needed?	No	Yes (usually)

WHY DO WE NEED FLIP-FLOPS?

- To **store data** — memory cells in CPUs and RAM
- To build **Registers** (multiple flip-flops working together)
- To build **Counters** (binary counting circuits)
- To **synchronize events** with a clock signal
- Used in **shift registers, accumulators**, and all sequential logic

02 What is a Gated Flip-Flop?

THE PROBLEM WITH BASIC LATCHES

A basic SR latch changes its output *immediately* whenever inputs S or R change. This is called being **level-sensitive / asynchronous** — it causes timing problems in real digital systems where you need everything to change at the *same controlled moment*.

Solution: Add an **Enable (EN) or Clock (CLK)** input — this is the "gate". The flip-flop only responds to inputs **when EN = 1**. This is called a **Gated Flip-Flop**.

COMPARISON: WITHOUT vs WITH GATE

Feature	Without Gate (Basic Latch)	With Gate (Gated Flip-Flop)
Responds to input	Any time (immediately)	Only when EN = 1
Timing control	None	Controlled by EN/CLK
Glitches/errors	Possible	Prevented
Operation type	Asynchronous	Synchronous

SIMPLE ANALOGY

Think of a gated flip-flop like a **security door with an access card reader**:

- **Inputs (S, R, D)** are like people trying to enter the room
- **EN/CLK signal** is the access card reader / gate controller
- When **EN = 0**: Door is LOCKED — no change happens regardless of inputs
- When **EN = 1**: Door is OPEN — inputs can change the output state

03 Gated SR Latch (SR Flip-Flop with Enable)

WHAT IS SR LATCH?

The **SR Latch** is the most basic flip-flop. **S = Set, R = Reset**. Two cross-coupled NAND (or NOR) gates form this latch.

- **S = 1, R = 0** -> Q = 1 (Set state)
- **S = 0, R = 1** -> Q = 0 (Reset state)
- **S = 0, R = 0** -> Q holds previous state (No Change / Memory)
- **S = 1, R = 1** -> INVALID / FORBIDDEN state ■

CIRCUIT LOGIC (GATED SR LATCH)

```
S -----[AND]------- S' -----[NAND]----- Q
           ^                    ^
EN ---------+                   |
           v                    |
R -----[AND]------- R' -----[NAND]----- Q'
When EN=0:  Both AND gates output 0  ->  NAND gates lock  ->  Q unchanged
When EN=1:  S and R pass through AND gates  ->  latch responds normally
```

TRUTH TABLE — GATED SR LATCH

EN	S	R	Q (next)	Q' (next)	Operation
0	X	X	Q (no change)	Q'	LATCH — disabled
1	0	0	Q (no change)	Q'	Hold / Memory
1	0	1	0	1	RESET
1	1	0	1	0	SET
1	1	1	?	?	■ INVALID / FORBIDDEN

■ **Exam Point:** S=1, R=1 is the **forbidden/invalid state** — both Q and Q' try to be 1 simultaneously, which violates the rule Q' = complement of Q. ALWAYS mention this in your exam answers!

CHARACTERISTIC EQUATION

Q(next) = S + R'.Q [Condition: S.R = 0 (SR must never both be 1)]

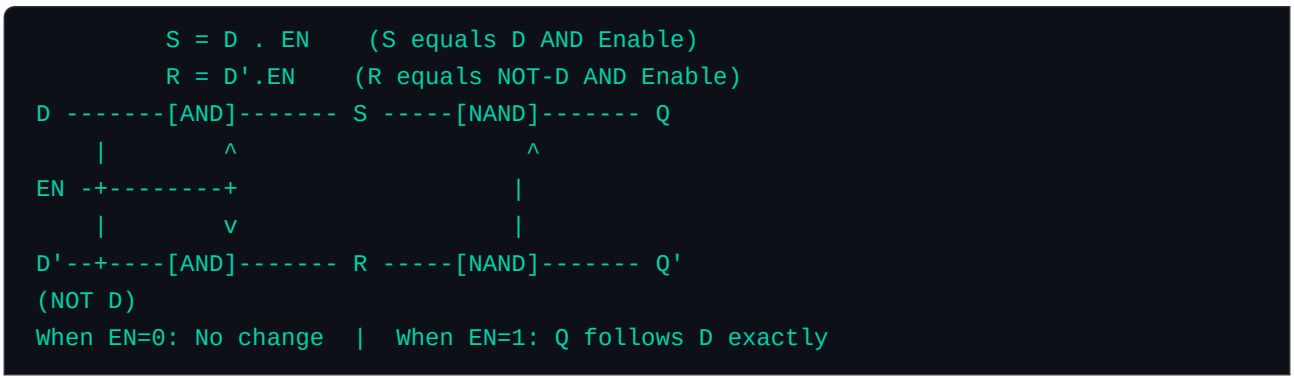
04 Gated D Flip-Flop (Data Flip-Flop)

WHY D FLIP-FLOP?

The SR flip-flop has the problem of **invalid state (S=R=1)**. The D Flip-Flop eliminates this completely by using only **one input (D)** and connecting R = D' internally. So S and R can NEVER both be 1 at the same time.

Golden Rule of D Flip-Flop: Whatever is at input D gets transferred to output Q when EN (clock) = 1. It is the simplest and most widely used flip-flop!

INTERNAL CONNECTIONS



TRUTH TABLE — GATED D FLIP-FLOP

EN (Clock)	D (Input)	Q (next)	Operation
0	X (don't care)	Q (unchanged)	Hold / Latch — no change
1	0	0	RESET — Q becomes 0
1	1	1	SET — Q becomes 1

■ **No invalid state!** The D flip-flop is the most widely used flip-flop in modern digital ICs (integrated circuits) because of its simplicity and no forbidden condition.

CHARACTERISTIC EQUATION

Q(next) = D (The next state simply equals the D input)

APPLICATIONS OF D FLIP-FLOP

- Building **Registers** — group of D flip-flops storing multiple bits
- **Data storage** in memory cells (SRAM, cache memory)
- **Shift registers** — serial data transfer and conversion
- Removing glitches — **synchronizing signals** to clock
- Input **debouncing** in digital systems

05 JK Flip-Flop (Clocked)

WHY JK FLIP-FLOP?

The JK flip-flop is an **improved SR flip-flop**. It eliminates the invalid state by making J=K=1 a valid condition — called **TOGGLE** (Q flips to its complement). J stands for "Jack" (Set), K stands for "King" (Reset).

J = Set, K = Reset (similar to S and R), but **J=K=1 is now VALID — it TOGGLES the output** (Q becomes Q')! This is what makes JK the "Universal Flip-Flop".

TRUTH TABLE — JK FLIP-FLOP

CLK Edge	J	K	Q (next)	Operation
^ (rising edge)	0	0	Q (no change)	Hold / No change
^ (rising edge)	0	1	0	RESET
^ (rising edge)	1	0	1	SET
^ (rising edge)	1	1	Q' (toggle)	■ TOGGLE — unique feature!

■ **Exam Trick:** JK solves SR's invalid state problem by making J=K=1 -> TOGGLE. Examiners often ask: "How does JK differ from SR?" Answer: The J=K=1 condition **toggles** output instead of giving undefined result.

CHARACTERISTIC EQUATION

$$Q(next) = J.Q' + K'.Q$$

RACE-AROUND CONDITION (IMPORTANT CONCEPT)

In a JK flip-flop with J=K=1, if the clock pulse is HIGH for too long, the output keeps toggling rapidly (Q→Q'→Q→...). This is called the **Race-Around Condition**.

Solution: Use **Master-Slave JK Flip-Flop** or **Edge-Triggered** design — this activates only on the clock edge (↑ or ↓), not during the entire HIGH period.

06 T Flip-Flop (Toggle)

WHAT IS T FLIP-FLOP?

The T (Toggle) flip-flop is derived from the JK flip-flop by connecting **J = K = T** (same input to both). It has only **one input T**.

- **T = 0** -> No change (Hold state — same as JK with J=K=0)
- **T = 1** -> Output TOGGLES (flips) on every clock edge

TRUTH TABLE — T FLIP-FLOP

CLK Edge	T	Q (next)	Operation
^ (rising edge)	0	Q (no change)	Hold — no change
^ (rising edge)	1	Q' (toggle)	TOGGLE — output flips

CHARACTERISTIC EQUATION

Q(next) = T XOR Q (XOR of T and current Q)

■ T flip-flops are used to build **binary counters** — connecting T=1 makes it toggle on every clock pulse, automatically counting in binary: 0→1→0→1...

DERIVING T FROM JK

```
JK Flip-Flop with J = K = T:
J -----+
       +-- [JK FF] ---- Q
K -----+
T ------> J
T ------> K       (connect same T to both J and K)
Result: T=0 -> Hold, T=1 -> Toggle  (T Flip-Flop behavior)
```

07 Comparison of All Flip-Flops

MASTER COMPARISON TABLE

Feature	SR	D	JK	T
No. of Inputs	S, R (2)	D (1)	J, K (2)	T (1)
Invalid State?	Yes (S=R=1)	No ■	No ■	No ■
Toggle Mode?	No	No	Yes (J=K=1)	Yes (T=1)
Hold condition	S=0, R=0	EN=0	J=0, K=0	T=0
Characteristic Eq.	S+R'.Q	D	J.Q'+K'.Q	TXORQ
Main use	Basic latch	Registers	Universal FF	Counters
Complexity	Simple	Simplest	Most complex	Simple

Most Important: JK flip-flop is called the **"Universal Flip-Flop"** because any other flip-flop (SR, D, T) can be implemented from it by appropriate input connections.

CONVERSION SUMMARY

To Get	From JK	Connection
SR Flip-Flop	JK FF	J=S, K=R (add condition S.R=0)
D Flip-Flop	JK FF	J=D, K=D' (connect complement of D to K)
T Flip-Flop	JK FF	J=K=T (connect same input to both J and K)

08 Registers

DEFINITION

A **Register** is a group of flip-flops that together store **multiple bits** of data. An **n-bit register** uses **n flip-flops** and can store an n-bit binary number.

Example: A **4-bit register** uses 4 D flip-flops -> can store 4 bits (like 1011 = decimal 11). Registers are used inside CPUs to hold data temporarily during processing.

4-BIT PARALLEL REGISTER (STRUCTURE)

```
D3--[FF3]--Q3    D2--[FF2]--Q2    D1--[FF1]--Q1    D0--[FF0]--Q0
             ^                ^                ^                ^
             +----------------+----------------+----------------+
                                CLK (one common clock to ALL flip-flops)
-> All 4 flip-flops share the SAME clock signal
-> All 4 bits are loaded SIMULTANEOUSLY in one clock cycle
-> Called PARALLEL LOAD because all bits load at the same time
```

TYPES OF REGISTERS

Type	Input Method	Output Method	Speed	Use Case
SISO Shift Reg.	Serial (1 bit/clock)	Serial	Slow	Data conversion / delay lines
SIPO Shift Reg.	Serial (1 bit/clock)	Parallel (all at once)	Medium	Serial-to-Parallel conversion
PISO Shift Reg.	Parallel (all at once)	Serial (1 bit/clock)	Medium	Parallel-to-Serial conversion
PIPO Register	Parallel (all at once)	Parallel (all at once)	Fast	CPU Registers, Buffers

4-BIT SHIFT REGISTER — OPERATION EXAMPLE

```
Initial state:    Q3=0  Q2=0  Q1=0  Q0=0   (register cleared)
Serial Input D = 1 (fed at Q3 each clock)
After CLK 1:  Q3=1  Q2=0  Q1=0  Q0=0   <- 1 entered from left
After CLK 2:  Q3=1  Q2=1  Q1=0  Q0=0   <- shifted right
After CLK 3:  Q3=1  Q2=1  Q1=1  Q0=0   <- shifted right
After CLK 4:  Q3=1  Q2=1  Q1=1  Q0=1   <- register full
Each clock pulse SHIFTS all bits one position to the right
New data enters from the LEFT (MSB side)
```

APPLICATIONS OF REGISTERS

- **CPU Registers** — AX, BX, CX, DX in 8086 microprocessor (16-bit registers)
- **Data storage** inside ALU (Arithmetic Logic Unit) during computation
- **Serial-to-parallel** conversion in communication interfaces (UART, SPI)
- Implementing **time delays** in digital signal processing
- Building **ring counters** and **Johnson counters**
- **Buffer registers** — temporary storage between fast CPU and slow memory

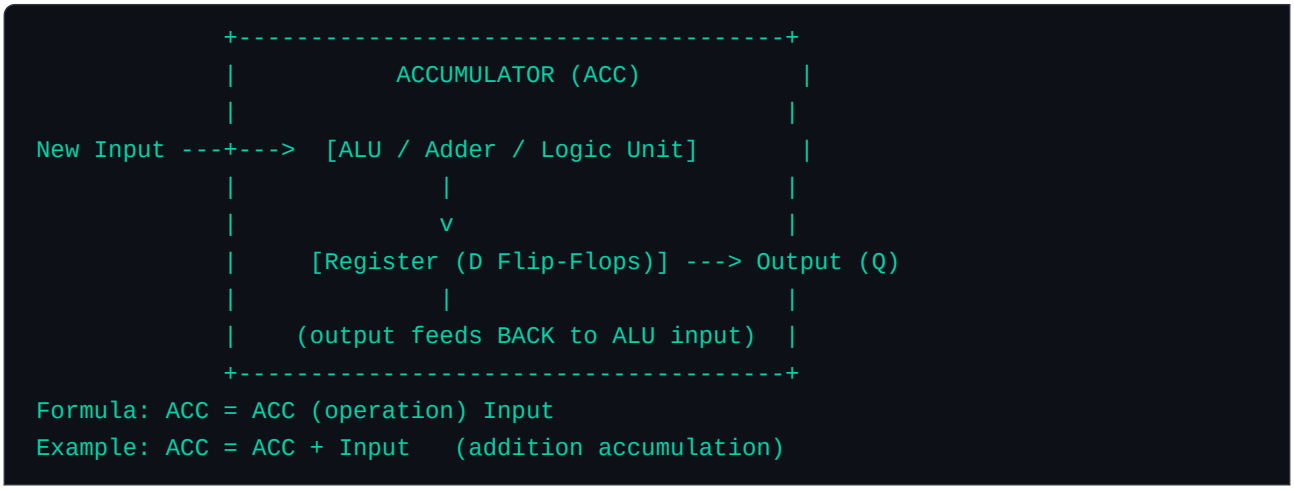
09 Accumulators

DEFINITION

An **Accumulator** is a **special register** that not only *stores* data but also *performs arithmetic and logic operations* on it. It is the most important register in any CPU/processor.

Key Idea: Accumulator = Register + ALU (Adder/Logic Unit). It **accumulates** the result of each operation back into itself for the next operation.

HOW ACCUMULATOR WORKS (BLOCK DIAGRAM)



STEP-BY-STEP OPERATION EXAMPLE

Step	Instruction	ACC Value	Explanation
Start	LOAD 5	5	Load value 5 into accumulator
Step 1	ADD 3	8	ACC = 5 + 3 = 8
Step 2	ADD 2	10	ACC = 8 + 2 = 10
Step 3	SUB 4	6	ACC = 10 - 4 = 6
Step 4	AND 7	6	ACC = 6 AND 7 = 6 (bitwise)
Step 5	STORE	6	Final result 6 saved to memory

REGISTER vs ACCUMULATOR — KEY DIFFERENCES

Feature	Register	Accumulator
Primary function	Stores data only	Stores data + performs operations
ALU connected?	No	Yes — tightly coupled with ALU
Feedback path?	No	Yes — output feeds back to ALU input
Operations	Read/Write only	ADD, SUB, AND, OR, XOR, etc.

Location in CPU	General purpose registers	Inside or adjacent to ALU
Example	BX, CX in 8086	AX (accumulator) in 8086

APPLICATIONS OF ACCUMULATOR

- Heart of every **CPU / Microprocessor** — all arithmetic goes through it
- Performing **repeated addition / subtraction** (loop operations)
- Computing **sum of arrays** — accumulate total in a loop
- **8085 has an 8-bit accumulator (A register)**, 8086 has AX (16-bit)
- **Digital Signal Processing (DSP)** — Multiply-Accumulate (MAC) operations
- **Cryptography** — hash computations and bitwise operations

10 Quick Revision & Exam Tips

ALL CHARACTERISTIC EQUATIONS

SR: Q(next) = S + R'.Q [Condition: S.R = 0]

D: Q(next) = D

JK: Q(next) = J.Q' + K'.Q

T: Q(next) = T XOR Q (XOR)

ONE-LINE DEFINITIONS (FOR SHORT ANSWER QUESTIONS)

- **Flip-Flop:** A bistable sequential circuit that stores 1 bit of data.
- **Gated FF:** A flip-flop with an Enable (EN) input that controls when state changes can occur.
- **SR FF:** A flip-flop with Set and Reset inputs; S=R=1 is the forbidden state.
- **D FF:** A flip-flop where output follows input D on each clock pulse; no invalid state.
- **JK FF:** Improved SR FF where J=K=1 causes toggle instead of invalid state; called universal FF.
- **T FF:** A single-input flip-flop that toggles output on every clock pulse when T=1.
- **Register:** Group of flip-flops storing multiple bits; n flip-flops = n-bit register.
- **Accumulator:** Special register with ALU that stores and accumulates arithmetic/logic results.

COMMON EXAM QUESTIONS

- Draw the truth table of Gated SR / D / JK flip-flop.
- Why is S=R=1 forbidden in SR flip-flop? How does JK flip-flop solve this?
- What is the characteristic equation of JK flip-flop? Derive it.
- Convert D flip-flop to T flip-flop (Answer: connect D = T XOR Q).
- Draw a 4-bit shift register and explain its operation with a timing diagram.
- Differentiate between SISO, SIPO, PISO, and PIPO shift registers.
- Differentiate between a register and an accumulator.
- What is the role of the accumulator in a microprocessor?
- Explain race-around condition in JK flip-flop and how to solve it.
- Why is the D flip-flop preferred over SR flip-flop in practice?

RECOMMENDED STUDY ORDER

Priority	Topic	Why Important

1st	Logic Gates (AND, OR, NOT, NAND, NOR)	Foundation — without this nothing makes sense
2nd	SR Latch (basic, no clock)	Simplest flip-flop — starting point
3rd	Gated SR -> Gated D Flip-Flop	Core syllabus — most exam questions
4th	JK Flip-Flop -> T Flip-Flop	Advanced FFs with toggle feature
5th	Registers (Parallel + All Shift types)	Uses multiple flip-flops together
6th	Accumulator	Practical application in processors

■ **Best Book Reference:** *Digital Design* by **M. Morris Mano** — Chapter 6 (Registers and Counters) and Chapter 9 (Memory and Programmable Logic). This book follows exactly the same sequence as your syllabus!

■ **Pro Tip for Exams:** Always draw the truth table first, then explain the invalid/special condition, then write the characteristic equation. This structure earns full marks in most universities.